

Department of Telecom and Networking

Course: CryptoGraphy

Term 1 | Year 3

Name: Chor Vanta

Project Report:

Title: SecureChat – Hybrid Cryptography Encrypted Messaging System

I. Introduction/ Background

1.1 Project Overview

SecureChat is a secure web-based messaging system that protects user messages using modern cryptographic techniques. The main goal of this project is to demonstrate how encryption can prevent attackers, including Man-in-the-Middle (MITM) attackers, from reading sensitive data.

Instead of sending messages in plaintext, SecureChat encrypts messages before they are sent to the server. Even if an attacker intercepts the network traffic, the message content remains unreadable.

1.2 Problem Statement

Traditional web applications often send data in plaintext or rely only on transport security. This creates several problems:

- Attackers can read messages using packet sniffing tools
- Password-based systems are vulnerable to phishing
- Man-in-the-Middle attacks can intercept communication

1.3 Proposed Solution

SecureChat solves these problems by using:

- **AES-256-GCM** for encrypting message content
- **RSA-2048** for securely exchanging encryption keys
- **HTTPS with a Self-Signed Certificate Authority (CA)** for transport security

This hybrid cryptographic approach ensures confidentiality and integrity of messages.

1.4 Cryptographic Concepts Used

In this project, the concept CryptoGraphic:

- Symmetric Encryption (AES)
- Asymmetric Encryption (RSA)
- Hybrid Encryption Model
- TLS / HTTPS
- Base64 Encoding

II. System Design / Architecture

2.1 System Architecture Overview

SecureChat consists of three main components:

1. **Client (Browser)**

- Encrypts messages using Web Crypto API
- Sends encrypted data to the server

2. **Server (Flask Backend)**

- Stores encrypted messages
- Decrypts messages only when requested by the receiver

3. **Certificate Authority (Self-Signed)**

- Issues TLS certificate for HTTPS communication

2.2 Encryption Workflow

Message Sending Process:

1. Sender writes a message
2. Browser generates a random AES-256 key
3. Message is encrypted using AES-GCM
4. AES key is encrypted using receiver's RSA public key
5. Encrypted data is sent to the server via HTTPS

Message Receiving Process:

1. Receiver requests message history
2. Server decrypts AES key using RSA private key
3. Server decrypts message using AES key
4. Plaintext message is sent to receiver

III. Implementation Details

3.1 Technologies Used

- **Frontend:** HTML, CSS, JavaScript
- **Backend:** Python (Flask)
- **Cryptography Libraries:**
 - Web Crypto API (client-side)
 - Python Cryptography Library
- **TLS:** OpenSSL self-signed CA

3.2 Key Components

chat.js (Client Side):

- Uses crypto.subtle API
- Generates AES-256-GCM keys
- Encrypts messages before sending
- Encrypts AES key using RSA-OAEP

chat.py (Server Side):

- Generates RSA key pairs per user
- Stores encrypted messages in JSON
- Decrypts messages securely
- Runs on HTTPS using self-signed certificates

Certificate Authority:

- CA key used to sign server certificate
- Server certificate enables HTTPS
- Protects against MITM attacks

3.3 Cryptographic Algorithms Used

Algorithm	Purpose
AES-256-GCM	Message encryption
RSA-2048 OAEP	AES key encryption
SHA-256	Hashing
TLS 1.2+	Transport security
Base64	Data Encoding
Web Crypto API	Browser Crypto

IV. Usage Guide

4.1 Requirement

- Python 3.9+
- OpenSSL
- Modern web browser

4.2 Installation Step

```
``` git clone https://github.com/VannTaa/Secure\_Chat\_Demo
cd Securechat
pip install -r requirements.txt
```
```

4.3 Running the Application

```
``` python chat.py ```
```

Open browser: <https://localhost:7000>

⚠ Browser will show warning due to self-signed certificate.

### 4.4 Example Result

- Messages are encrypted in transit
- Burp Suite shows only ciphertext
- Plaintext visible only to authorized receiver

## V. Conclusion and Future Work

### 5.1 Conclusion

SecureChat successfully demonstrates the use of hybrid cryptography to protect message confidentiality. The system prevents attackers from reading messages even when network traffic is intercepted.

### 5.2 Future Work

- Full client-side decryption (true E2EE)
- Digital signatures for message authenticity
- User authentication system
- Secure key storage using HSM or TPM

## VI. References

1. [How to set up Self-Signed Certificate](#)
2. [RFC 4106 – GCM Mode](#)
3. [NIST AES Standard](#)
4. [Mozilla Web Crypto API Documentation](#)
5. [OWASP Cryptographic Storage Cheat Sheet](#)